

Jaunpur Footwear Deployment and Setup Guide

Local setup, production configuration, release sequence, recovery and verified QA

Prepared 8 October 2026 | Repository review | Branch: sartaj

Purpose and deployment status

This guide matches the repository reviewed on 8 October 2026. Local development uses SQLite. Production settings require PostgreSQL, Redis/Celery and durable media storage. A production WSGI server is not pinned in requirements.txt; select and add it for the chosen host before deployment. No live production environment is claimed.

1. Local setup on Windows PowerShell

Install Python 3.11 and Git. From the repository root, run:

```
py -3.11 -m venv .venv
./venv/Scripts/Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Open <http://127.0.0.1:8000/>. .env.example is reference only and Django does not load it automatically. Set values in PowerShell or a deployment secret store. Back up any existing local database before applying migrations.

2. Runtime and production services

Component	Production requirement
Python and packages	Python 3.11 is used in CI. requirements.txt includes Django 5.2, PostgreSQL driver, DRF, Celery/Redis, Pillow, S3 storage and ReportLab.
Web server	WSGI target footwear.wsgi:application. Add/pin a production server and host-specific start command; runserver is development-only.
Database	PostgreSQL via DJANGO_DATABASE_URL; production settings request SSL when sslmode is omitted.
Queue and scheduler	Redis via CELERY_BROKER_URL; supervise Celery worker and Beat separately.
Media and static files	S3-compatible media by default in production; set bucket/region/credentials, collect static files and configure static serving.
Domain and secrets	HTTPS, exact allowed hosts/CSRF origins, secret store, logs, health monitor and database/media backups.

3. Environment settings

Set secrets through the hosting platform. Production requires DJANGO_ENV=production, DJANGO_DEBUG=false, DJANGO_SECRET_KEY, DJANGO_ALLOWED_HOSTS, PostgreSQL DJANGO_DATABASE_URL, CELERY_BROKER_URL and an S3 bucket unless another supported media backend is deliberately selected.

Settings	Use
DJANGO_ENV, DJANGO_DEBUG	Select environment; production debug must be false.
DJANGO_SECRET_KEY	Strong random production secret kept outside Git.
DJANGO_ALLOWED_HOSTS, DJANGO_CSRF_TRUSTED_ORIGINS	Exact public domains and HTTPS origins.

Settings	Use
DJANGO_DATABASE_URL	PostgreSQL connection URL with protected credentials.
CELERY_BROKER_URL	Redis broker for worker and Beat.
DJANGO_MEDIA_STORAGE, AWS_STORAGE_BUCKET_NAME, AWS_S3_REGION_NAME, AWS_S3_ENDPOINT_URL	S3-compatible media setup, bucket, region and optional endpoint/credentials.
EMAIL_BACKEND, EMAIL_HOST, EMAIL_PORT, EMAIL_USE_TLS, EMAIL_HOST_USER, EMAIL_HOST_PASSWORD, DEFAULT_FROM_EMAIL	Real email service; local default prints messages to console.
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET	Payment API and separate webhook signing secret.
WHATSAPP_GRAPH_API_VERSION, WHATSAPP_PHONE_NUMBER_ID, WHATSAPP_ACCESS_TOKEN, WHATSAPP_ORDER_TEMPLATE	Meta API, approved template and customer opt-in.
MARKETPLACE_TOKEN_ENCRYPTION_KEY	Stable encryption secret for seller tokens; back it up securely.
FLIPKART_CLIENT_ID, FLIPKART_CLIENT_SECRET, FLIPKART_REDIRECT_URI	Registered Flipkart app and seller callback.
AMAZON_APPLICATION_ID, AMAZON_LWA_CLIENT_ID, AMAZON_LWA_CLIENT_SECRET, AMAZON_REDIRECT_URI	Registered Amazon app, seller consent, roles and region endpoint.
ONDC_PARTICIPANT_ADAPTER	Optional dotted path to a participant-specific adapter; not a full integration.
ORDER_STOCK_RESERVATION_MINUTES	Positive reservation expiry duration; default 15 minutes.
BACKUP_HEALTHCHECK_PATH, BACKUP_MAX_AGE_HOURS, BACKUP_RESTORE_CHECK_INTERVAL_DAYS, OPERATIONS_ALERT_EMAILS	Backup freshness/restore checks and alert recipient setup.

4. Release sequence

- Provision staging PostgreSQL, Redis, media storage, HTTPS, secrets, logs and backup destination.
- Add and pin a production WSGI package and the provider start command.
- Run `python manage.py check --deploy` with production settings and resolve findings.
- Back up data; inspect showmigrations; run migrate as a controlled release step.
- Run `collectstatic --noinput` and verify public static assets and uploaded media.
- Start WSGI, Celery worker and Celery Beat under supervised processes.
- Rehearse checkout, webhook, cancellation/refund review, seller permissions, health alert and recovery in staging.
- Promote only after backup restore and rollback/recovery steps are rehearsed.

5. Background jobs

Beat schedules unpaid Razorpay reservation expiry every minute, authorized marketplace sync every 30 minutes, and production operations checks every 15 minutes. Supervise worker and Beat and monitor job failures.

```
celery -A footwear worker -l INFO
celery -A footwear beat -l INFO
```

6. Staging rehearsal

- Browse, variants, delivery coverage, fulfillment slot, multi-shop checkout, cancellation and return.
- On PostgreSQL, have concurrent buyers claim the final stock unit and confirm no oversell.

- Use Razorpay test credentials for success, failure, bad/duplicate signature, amount mismatch, delayed callback and refund review.
- Confirm each seller is scoped to its data and staff actions appear in audit history.
- Upload an image, verify it after redeploy, simulate provider timeout and inspect task failure/retry visibility.
- Back up database/media and restore to a separate environment; record recovery time and gaps.
- Use approved provider sandboxes and seller consent; leave unavailable connections disabled.

7. QA performed on 8 October 2026

Check	Result	Scope and limit
Django system check	Pass	No issues reported.
Migration check	Pass	makemigrations --check --dry-run found no changes.
Automated tests	37 passed	Focused Django tests; mock failure logs are expected negative-path checks.
GET route load	500/500 HTTP 200	25 virtual clients; local runserver/SQLite; about 41.2 RPS, p50 182 ms, p95 1,825 ms, max 2,457 ms.
Concurrent checkout	12/12	Synthetic customers/stock on disposable local PostgreSQL; no real payment or user data.

The GET run covered storefront/catalog, shop/area, APIs, health and public metadata. Both runs used one development machine. They are regression checks only, not a production capacity promise or proof of real provider, worker, storage or delivery-device behavior.

8. Launch decision

The feature scope is ready for a portfolio demonstration after a rehearsed walkthrough. Before public launch, pin the production server, pass production checks, verify staging deployment and restore, rehearse provider paths, and assign owners for alerts and support. A controlled pilot can follow those gates; no extra broad feature list is needed before interviewing.

9. Repository references

- docs/production-operations.md
- docs/ondc-participant-adapter.md
- .env.example
- README.md